

数理推論タスクにおける学習・推論パイプラインの構築

本郷颯人¹ 加地翔太² 小谷真士³ 嶋中雄大⁴
宮川裕貴⁵ 中島悠樹¹ 安孫子リク¹ 松田公慶⁶
¹東京大学 ²独立研究者 ³大阪公立大学
⁴芝浦工業大学 ⁵中京大学 ⁶筑波大学
連絡先: 本郷颯人

概要

本研究は、「第2回大規模言語モデルのファインチューニング技術と評価 (FT-LLM 2026)」チューニングコンペティションの数学タスクにおける取り組みを報告する。ベースモデル llm-jp-4-8b-instruct を出発点とし、32 層から 48 層への深さ拡張による 12B 級モデル化、最大系列長 16384 の長文 CoT によるフルパラメータ SFT、および GRPO による強化学習を組み合わせることで数理推論能力の向上を図った。最終提出モデル open-instruct-grpo-fast-12B-1725 は、推論時にはモデル単体と多数決によるテストタイムスケールリングを用い、dev.jsonl で cons@40 = 0.96 を達成した。また、TIR や GPT-OSS judge はオフライン評価・分析に利用し、提出システムには含めない構成とした。

1 はじめに

近年、大規模言語モデル (LLM) は自然言語処理の多岐にわたる分野で目覚ましい成果を挙げている。しかし、厳密かつ多段階の論理展開が要求される数理推論タスクにおいては、依然としてハルシネーションや計算ミスが発生しやすく、LLM にとって最も難易度の高い領域の一つとされている。この課題を克服するため、良質な Chain of Thought (CoT) データを用いた教師ありファインチューニング (SFT) や、人間の評価またはルールの下でモデルを最適化する強化学習 (RLHF 等) といったファインチューニング技術の重要性が急速に高まっている。

本研究は、「第2回大規模言語モデルのファインチューニング技術と評価 (FT-LLM 2026)」チューニングコンペティションの数学タスクにおける取り組みを報告するものである。本タスクの課題設定は、80 億パラメータのベースモデル llm-jp-4-8b-instruct を対象に、日本語で出題される日本の中学校・高等学校レベルの数学問題に対する推論能力を向上させることである。比較的小規模なモデルにおいて、限られた計算資源の中でいかにして高度な数理的思考能力を獲得させ、正確な解答を導き出すかが主なターゲットとなる。

この課題に対し、本研究では学習から推論に至る一連のパイプラインを構築した。まず、Python コード実行環境を用いた厳密な正答検証に基づくデータキュレーションを行い、質の高い学習データを

厳選した。次に、複雑な推論過程やコード生成に対応できるよう長文コンテキストを考慮した SFT を実施し、基礎的な解答能力を付与した。さらに、学習の安定化とフォーマット遵守、および推論ステップの最適化を目的として、GRPO (Generative Reward Policy Optimization) による強化学習を適用した。推論時には、多数決 (Majority Voting) によるテストタイムスケールリングを最終提出系として採用し、TIR (Tool Integrated Reasoning) はデータクリーニングおよびオフライン分析・評価系で検証した。本稿では、これらの統合的なアプローチの詳細とその評価結果について述べる。

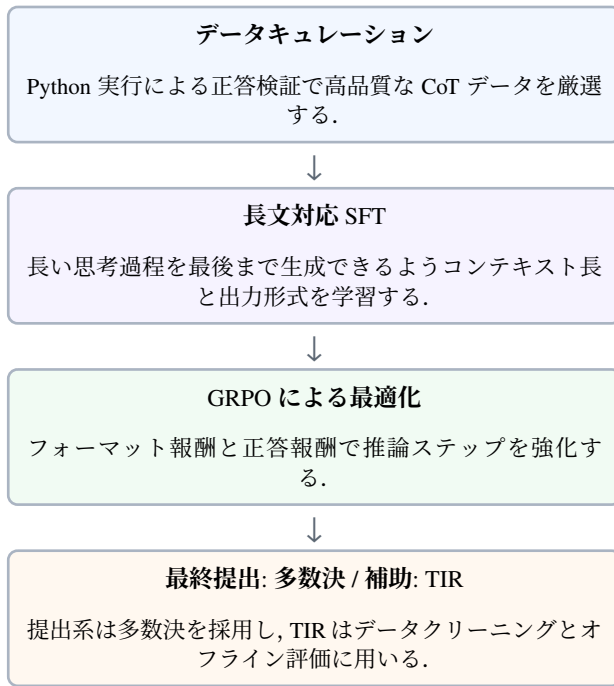


図 1: 提案手法の全体像

提案手法は、学習時の改善と推論時のスケールリングを段階的に積み上げる構成であり、その全体像を図 1 に示す。

2 関連研究

2.1 LLM における数理推論能力の向上

大規模言語モデル (LLM) に複数ステップを要する複雑な数学問題を解かせるための画期的な手法として、中間的な推論過程を言語化して出力させる

Chain of Thought (CoT) プロンプティング [1] が提案され、モデルの数理推論能力が劇的に向上することが実証された。さらに、強力なクロードモデルが持つ推論能力を比較的小規模なモデルに蒸留・移植するための教師ありファインチューニング (SFT) も盛んに研究されている。SFT の成否はデータセットの質と量に大きく依存するため、大規模な合成データの構築が重要なアプローチとなっている。その代表例として、Mixtral モデルを用いて 180 万件もの数学的推論データと複数パターンの正答パスを生成した OpenMathInstruct-1 [2] などが挙げられる。本研究の初期段階においても、こうした先行研究に倣い、厳選された合成データセットを用いた SFT を実施している。

2.2 推論プロセスに対する強化学習

SFT による模倣学習の限界を超え、モデル自身が自律的に正しい推論パスを探索するための手法として、強化学習 (RL) の適用が近年著しい成果を上げている。DeepSeekMath 等で導入された GRPO (Generative Reward Policy Optimization) [3] は、同一プロンプトから生成された複数の出力群 (グループ) 内で報酬を正規化することにより、従来の PPO (Proximal Policy Optimization) で必要とされた価値モデルを省略し、メモリ消費を抑えつつ安定した最適化を可能にした。さらに、数学やプログラミングのように正解が客観的に検証可能なタスクにおいては、人間の評価の代わりにルールベースの正誤判定を報酬として与える RLVR (Reinforcement Learning with Verifiable Reward) [4] が極めて有効であることが示されている。本研究でも、これらの強化学習の枠組みを採用し、フォーマットの遵守と推論ステップの最適化を図っている。

2.3 テストタイムスケールリング

近年の LLM 研究においては、学習時の計算資源を増やすだけでなく、推論時の計算資源をスケールリングさせることで精度を劇的に向上させるパラダイムが注目を集めている。その基礎となる強力な手法が、サンプリング温度を上げて多様な推論パスを複数生成し、その中で最も多く導出された解答を最終出力とする多数決 (Majority Voting / Self-Consistency) [5] である。また直近の動向として、推論時の思考ステップを意図的に拡張させ、モデルに自己検証や修正を促すアプローチも提案されている。例えば s1 [6] などの研究では、推論時にモデルの生成を強制的に延長させる Budget Forcing 等のシンプルな手法によって推論時の計算量をコントロールし、テストタイムスケールリングの恩恵を効率的に引き出している。本研究のシステム構築においても、これらの知見を応用し、提出系では多数決によるテストタイムスケールリングを採用し、TIR は補助的な分析・評価系として検証した。

3 数理推論に向けた学習パイプライン (提案手法 1)

本章では、ベースモデルである 80 億パラメータの LLM (llm-jp-4-8b-instruct) に対し、高度な数理推論能力を付与するための学習パイプラインについて述べる。本アプローチは、厳格なデータキュレーション、長文コンテキストを考慮した教師ありファインチューニング (SFT)、そして強化学習 (GRPO) による推論プロセスの最適化という 3 つの段階で構成される。

1. データキュレーション

処理: Python 実行で検証し、低品質データを除外する。
狙い: ノイズの少ない推論データを構築する。

2. SFT

処理: 長文 CoT と出力フォーマットをフルパラメータ学習する。
狙い: 中間思考と最終解答を安定して生成させる。

3. GRPO

処理: フォーマット報酬と正答報酬で推論を最適化する。
狙い: 深い推論探索と正答率の両立を図る。

表 2: 学習パイプラインの各段階

表 2 は、本研究の学習パイプラインを 3 段階に整理したものである。

3.1 データセットの構築とフィルタリング

LLM に複雑な数理推論の過程 (Chain of Thought; CoT) を学習させるため、大規模な合成データセットの収集と厳密な選別を行った。最終的に採用した主なデータセットは、qa_verify_cot_new_6M_v6 (約 560 万件)、qa_verify_2M_v5 (約 175 万件)、および TIR 専用の qa_verify_tir_1.3M_v0 (約 130 万件) である。後者は、実行可能コードのみを通したフィルタリング済みデータとして利用した。

しかし、既存の合成データセットには、推論過程における論理的飛躍や計算ミスといったノイズが多く含まれている。そこで本研究では、TIR (Tool Integrated Reasoning; ツール統合推論) のパラダイムを応用し、Python コード実行のサンドボックス環境を用いた正答検証パイプラインを構築した。具体的には、収集した問題に対してモデルに Python コードを生成させ、それをサンドボックス環境で実行して得られた出力と正解ラベルを自動で照合する。この際、複数回サンプリングを行った際の正答基準 (例: pass@8 での正解有無) を設け、一貫して正しい答えを導き出せない品質の低いデータや、実行時エラーを引き起こすデータを除外するフィルタリング戦略を適用した。

一方で、ライセンスおよび競技ルールの観点から、OpenMathInstruct-2, MATH, PRM800K, competition_math, DeepScaleR-Preview-Datasetなどは採用しなかった。特に OpenMathInstruct-2 は Llama-3.1-405B 蒸留由来の可能性があるが、また MATH 系列や AIME/AMC 由来データは著作権上の懸念が大きいため、不採用とした。評価用に利用した MATH-500-ja については、非公開テストの代理ベンチマークとして扱い、SFT データ生成時にはホワイト/ブラックリスト運用により使用不可データを弾く方針を採った。

3.2 教師ありファインチューニング (SFT)

構築した高品質データセットを用い、ベースモデル llm-jp-4-8b-instruct に対する SFT を実施した。最終的に比較した SFT モデルは llm-jp-4-8b-instruct-sft-expand-checkpoint-1900 であり、ベースモデルを 12B 規模へ層拡張した上で、最大系列長 16384 の長文 CoT データに対するフルパラメータ SFT を行った。深さ拡張は、OpT-DeUS.py の index_mapping に基づき、前半 16 層をそのまま保持しつつ、元の後半層を奇数番目へ再配置し、偶数番目の新規層 (16, 18, ..., 46) を隣接層間の Optimal Transport 補間で初期化する形で 48 層へ拡張した。さらに、挿入層の self_attn.o_proj と mlp.down_proj はゼロ初期化されている。学習環境には ABCI 3.0 の 1 ノード (NVIDIA H200 × 8 GPU) を用いた。TIR 用データセットに対する学習では 2 エポックを採用した。

数理推論タスクでは、複雑な問題になるほど中間的な思考プロセスが長大化する。事前実験において、デフォルトの短いコンテキスト長では推論の途中で生成が打ち切られる (Truncation) 問題が発生し、モデルが最終的な解答に到達できず学習が不安定になる現象が確認された。これに対処するため、学習コンフィグにおいてコンテキスト長を 16384 まで拡張して SFT を行い、長大な思考プロセスを最後まで破綻なく生成しきれるようにモデルを適応させた。公開されている長文 SFT 設定ファイルでは、`learning_rate=4.0e-5`, `per_device_train_batch_size=1`, `gradient_accumulation_steps=32`, `num_train_epochs=1`, `optim=adamw_torch`, `lr_scheduler_type=cosine_with_min_lr`, `warmup_ratio=0.03`, `max_grad_norm=0.2` が用いられている。データについては qa_verify_2M_v5 を中心に少なくとも 175 万件規模、最終的には 250 万件以上の CoT データを投入した。またプロンプト設計において、モデルに思考プロセスと最終的な解答を明確に分離させるためのフォーマット (例: `<think>...</think>` タグの使用や、`\boxed{}` による解答の囲み) を指定し、推論構造の定式化を促した。

3.3 強化学習 (GRPO) による最適化

SFT によって基礎的な解答能力とフォーマットの遵守能力を獲得したモデルに対し、推論ステップのさらなる最適化と自律的な探索能力 (Deep Thinking) の向上を目的として、GRPO を適用した。最終的な GRPO モデルは open-instruct-grpo-fast-12B-1725 であり、SFT モデル llm-jp-4-8b-instruct-sft-expand-checkpoint-1900 を初期ポリシーとして、Open-R1 フレームワーク上で 1725 ステップ学習した。対応する HF ブランチ名は grpo_fast_3_1770120411 である。

GRPO における報酬関数は、主に 2 つの要素から設計した。1 つ目は、`\boxed{}` などの指定フォーマットを正しく出力できているかを評価するフォーマット報酬であり、2 つ目は、Math-Verify 等の数式パーサーを用いて最終的な解答が正解と一致しているかを判定する正答 (Outcome) 報酬である。実装は open-r1 (verl ベース) に基づき、チェックインされている 12B 用ジョブスクリプトでは `num_unique_prompts_rollout=8`, `num_samples_per_prompt_rollout=32`, `response_length=7168`, `pack_length=8192`, `learning_rate=1e-6`, `beta=0.00`, `seed=3`, `num_epochs=1` が指定されている。この設定では rollout バッチサイズは $8 \times 32 = 256$ となる。

GRPO の学習過程においては、モデルがより長く深い推論を探索するようになるため、生成トークン数が急増してメモリ不足 (OOM: Out of Memory) を引き起こしやすくなる。これに対し、本研究では学習環境における生成と最適化のノード・GPU 割り当ての調整、コンテキスト長拡張を施した SFT チェックポイントからの学習再開、および長文生成時のペナルティや Wait パラメータによる調整といった学習安定化の工夫を取り入れた。特に GRPO では、おおむね 700 ステップまでは qa_verify_2M_v5 を用い、それ以降はスループット低下を防ぐため qa_verify_cot_new_6M_v6 に切り替えるカリキュラム的運用を行った。また、チェックインされている 12B 用ジョブスクリプトでは学習終盤の Temperature は 1.018, clip_higher は 0.272 に設定されている。12B モデルの rollout バッチサイズは 256 に固定されており、vLLM の KV キャッシュ制約を考慮した設計になっている。推論時の Wait 値については 0, 1, 2, 3 を比較し、cons@40 が最も高かった Wait=0 を開発評価および提出系の既定設定として採用した。

4 推論・評価システムの実装 (提案手法 2)

本研究では、学習フェーズでのモデル性能向上 (Train-time Scaling) に加え、推論フェーズにおける計算資源のスケーリング (Test-time Scaling) を通じて最終的な正答率を最大化するシステムを構築し

た. 提出系では, サンプリングによる多数決 (Majority Voting / Self-Consistency) を中核とする推論パイプラインを実装し, 外部ツールを用いた回答検証メカニズムであるツール統合推論 (TIR) は, データクリーニングおよびオフライン分析・評価系として切り分けて運用した.

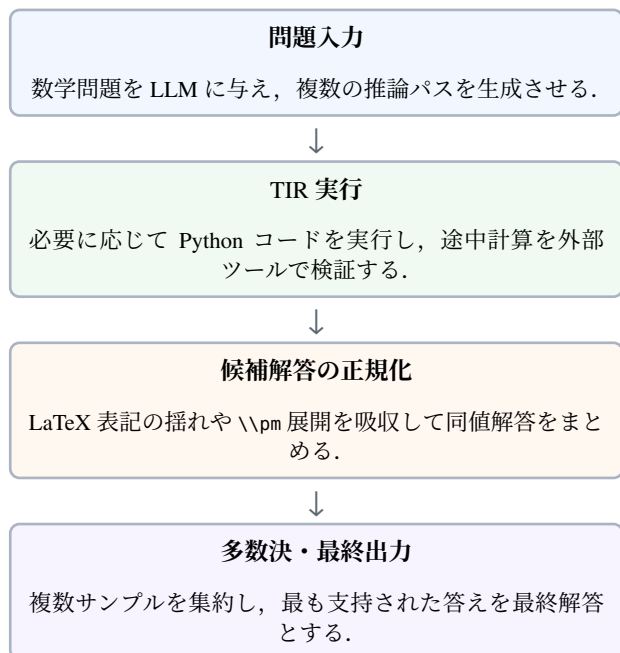


図 3: 推論・評価システムの流れ

図 3 は, 提出系の多数決推論と, 補助系である TIR ベース評価を含む推論・評価フローの全体像を示している.

4.1 ツール統合推論 (TIR)

LLM 単独での推論では, 複雑な四則演算や高次方程式の求根などにおいて, 論理展開は正しいものの途中の計算ミスによって最終的な解答を誤るケースが散見される. この課題を解決するため, LLM によるコード生成と安全なサンドボックス環境でのコード実行を組み合わせた TIR システムを導入した.

本システムでは, NeMo-Skills に近いアーキテクチャを参考に, Singularity コンテナ上で vLLM サーバーとサンドボックスを共存させた. 推論を制御するオーケストレーター, LLM, そして分離された実行環境が相互に連携し, LLM が必要に応じて生成した Python コードをサンドボックスで実行する. 実行結果 (標準出力やエラーログ) は再び LLM にフィードバックされ, LLM はその客観的な計算結果に基づいて推論を補正・続行する.

ただし, TIR 付き GPT-OSS は dev.jsonl 100 問に対する独自評価で pass@1 = 0.86 と高い性能を示した一方, 無限ループや統合コスト, 出力の不安定化といった課題が残った. この評価は NeMo-Skills のサンドボックス環境上で Math-Verify を用いて算出した

ものであり, 中 1: 93.3%, 中 2: 94.4%, 中 3: 75.0%, IA: 80.0%, IIB: 88.4%, IIIC: 80.0% という内訳であった. そのため, TIR はオフライン評価・分析およびデータクリーニングには活用したが, 最終提出システムには組み込まなかった.

4.2 多数決 (Majority Voting) による解答抽出

テストタイムスケーリングの恩恵を最大限に引き出すため, 単一の推論パス (Greedy Decoding) による解答抽出 (pass@1) ではなく, 多数決アルゴリズムを推論システムに組み込んだ. コードベースの評価用スクリプトでは, 推論時のサンプリング温度を 0.7, 最大生成長を 16384, サンプル数を 40, wait_count を 0 とし, 同一の入力に対して多様な思考プロセスを伴う複数回答を生成させ, その中で最も多く導出された解答を最終的な出力として採用した. 最終提出では cons@40 を採用し, 100 問の dev.jsonl に対して約 960 秒で推論を完了した.

多数決を機能させる上で最大の課題となったのが, LLM が生成する LaTeX 形式の数式表記の揺れである. 例えば, 数学的には完全に等価な数式であっても, 空白やカンマの有無, あるいは \pm の展開といった出力フォーマットの揺れによって, 単純な文字列一致では別々の解答として集計されてしまう問題が発生した.

この集計割れを防ぐため, 抽出された解答の文字列に対し, 不要な空白の削除や特定の記号 (\pm 等) を正規化する後処理スクリプト replace_pm.py を実装した. さらに, 単純なルールベースでは吸収しきれない複雑な数式の等価性判定やパース処理に対しては, GPT-OSS-20B や ChatGPT API をオフライン評価用ジャッジとして用いた. これらの外部 judge は評価・分析用途に限定し, インターネット遮断下で動作する最終提出システムには含めていない.

5 実験と評価

5.1 実験設定

本研究で構築した学習・推論パイプラインの有効性を検証するため, 2 種類のデータセットを用いて評価を行った. 1 つ目は, コンペティション運営から提供された開発用データセット (dev.jsonl, 100 問), 2 つ目は, より難易度が高く汎化性能を測るための外部ベンチマークである MATH-500 の日本語翻訳版 (MATH-500-ja) である. dev.jsonl の内訳は, 中 1: 15 問, 中 2: 18 問, 中 3: 16 問, 数 IA: 15 問, 数 IIB: 26 問, 数 IIIC: 10 問である.

評価指標としては, 単一の推論結果における正解率 (pass@1) と, k 回のサンプリング結果から多数決を行って得られた最終解答の正解率 (cons@k) を用いた. なお cons@1 は定義上 pass@1 と等価であり, 本稿の表では cons@1 に統一して示す. 評価時の一致判定には Math-Verify による Normalized Match を用い, 必

要に応じて `replace_pm.py` による文字列正規化を適用した。なお、コードベースの評価設定ではサンプリング時の `Temperature` は 0.7, 最大生成長は 16384, サンプル数は 40, `wait_count` は 0 である。推論の実行環境は、本番環境を想定した ABCI 3.0 の 1 計算ノード (NVIDIA H200 GPU 8 基搭載, RAM 2TB) である。100 問に対する `cons@40` 推論は約 960 秒で完了し、500 問換算でも約 80 分であり、競技制約の 500 分以内に十分収まる。

5.2 定量評価結果

ベースモデル (`llm-jp-4-8b-instruct`), 提案する SFT モデル (`llm-jp-4-8b-instruct-sft-expand-checkpoint-1900`), および GRPO モデル (`open-instruct-grpo-fast-12B-1725`) の性能比較を行った。まず `dev.jsonl` における結果を見ると、ベースモデルの `cons@1=0.440` に対し、長文対応の SFT を施したモデルでは 0.840 へと大きく向上した。さらに、GRPO を適用した最終モデルでは `cons@1=0.870` に達し、SFT から GRPO への移行によっても改善が確認された。

次に、テストタイムスケーリング (多数決) の効果を検証した結果、GRPO モデルにおいて `cons@20=0.950`, `cons@40=0.960` と、サンプリング数を増やすごとに精度が単調増加した。一方、`MATH-500-ja` においてもベースモデルの `pass@1=22.2%` に対し、SFT モデルで 38.6%, GRPO モデルでは 71.0% と大幅な改善が見られ、本提案手法が未知の難問に対しても一定の頑健性を持つことが示された。`dev.jsonl` は 100 問であるため、1 問差が 1% に相当する点には留意が必要であるが、`grpo-step1725 (wait=0)` においては `pass@1=0.87`, `cons@40=0.95~0.96` の範囲で安定した性能を示した。この幅は、評価スクリプト側の微細な正規化パッチや後処理の有無による再評価差に起因する。

指標	Base	SFT	GRPO
<code>cons@1</code>	0.440	0.840	0.870
<code>cons@20</code>	0.660	0.920	0.950
<code>cons@40</code>	0.680	0.930	0.960
<code>cons@80</code>	0.720	0.920	-
<code>cons@160</code>	0.770	0.920	0.950

表 1: `dev.jsonl` における主要なアブレーション結果

Wait	0	1	2	3
<code>cons@40</code>	0.95–0.96	0.92	0.93	0.93

表 2: Wait パラメータの比較 (GRPO, `cons@40`)

性能比較の要点を表 1 にまとめる。列数を抑えた転置表とすることで、Base, SFT, GRPO の差分が読み取りやすくなった。また Wait 値別の比較を表 2 に分離した。`cons@40` は `Wait=0` が最も高く、最終的にこの設定を採用した。

5.3 定性評価とエラー分析 (アブレーション)

定量的に高いスコアを達成した一方で、いくつかの技術的課題とモデルの傾向が定性分析から明らかになった。

評価スクリプトのフォーマット不一致問題

推論結果の評価において、ルールベースのパarser (`Math-Verify` 等) が正解を不正解 (False Negative) として誤判定するケースが散見された。具体的には、模範解答 $\frac{1}{\sqrt{97}}$ に対して出力が $\frac{1}{12} - \frac{\sqrt{97}}{12}$ のように展開される場合、あるいは $\sqrt{}$ の前後の半角スペースやカンマの有無が異なる場合である。また、模範解答が $(x,y) = (4,2)$ であるのに出力が 4 のみであったような、形式上の欠落も見られた。本システムではこの揺れを吸収するため、多数決の集計前に $\sqrt{}$ の展開や空白除去を行う後処理スクリプトと、GPT-OSS を用いたジャッジモデルを導入することで、正当な解答を適切に評価・抽出できるようになった。

コンテキスト長と Wait パラメータの影響

Wait パラメータの影響を調査した結果、`cons@40` では `Wait=0` が最も高く、`Wait=1,2,3` はいずれもそれを下回った。`wait_count` を増やすと、回答前に “Wait” を追加して再生成させるため思考を引き延ばせる一方、出力長の増大によってトークン制限 (16384 トークン等) に近づき、かえって精度が低下するケースが見られた。したがって、本タスクでは長文化の強制よりも、長い最大生成長と多数決を組み合わせた `Wait=0` の設定が最も安定であった。

分野別の得意・不得意

カテゴリ別および単元別 (`by_category_stats`, `by_unit_stats`) の正答率を分析した結果、中学レベルの数学 (中 2: 18 問中 72.2%, 中 1: 15 問中 66.6%) や、高校数学の一部である「微分法・積分法」(75.0%) において高い推論能力を示した。一方で、「数学 IIB」(26 問中 23.0%) や「数学 IA」(15 問中 26.6%) 全体ではスコアが低迷しており、特に「三角関数」(0.0%) や「指数・対数」(0.0%) といった、純粋なテキストベースの思考だけでは代数的な式変形が極めて複雑になる分野を苦手としている傾向が浮き彫りになった。

6 おわりに

本稿では、「第 2 回大規模言語モデルのファインチューニング技術と評価」チューニングコンペティションの数学タスクにおいて、80 億パラメータのベースモデル (`llm-jp-4-8b-instruct`) の数理解論能力を向上させるための統合的なアプローチを提案し、その有効性を実証した。

本研究の結論として、比較的小規模なモデルであっても、学習から推論に至るパイプラインを精緻に設計することで、高度な数理的思考能力を獲得できることが明らかになった。具体的には、外部ツールを用いた厳密な正答検証による学習デー

タのキュレーション、長文コンテキストに対応した教師ありファインチューニング (SFT)、そしてフォーマットと正答に基づく報酬を用いた強化学習 (GRPO) を組み合わせることで、モデルの自律的な推論探索能力を引き出すことに成功した。さらに、最終提出系ではモデル単体とサンプリングによる多数決 (Majority Voting) を採用し、オフラインの評価・分析では LLM ジャッジや TIR を補助的に用いることで、未知の数学問題に対しても高い正解率を達成した。

今後の課題としては、より複雑で長大な推論を必要とする問題に対応するため、学習・推論の双方におけるさらなる長文コンテキストへの適応と、それに伴う Truncation 問題の完全な解決が挙げられる。また、現状はデータ生成や後処理に留まっている TIR を、推論サイクルの中にシームレスに組み込み、LLM が自律的にコードを実行して自己修正を行うプロセスの最適化も重要な検討事項である。最後に、本研究で構築した高品質データによる SFT と GRPO、そして推論スケーリングのパラダイムは、数学タスクに留まらず、プログラミングや論理パズルといった他の推論集約型タスクへの応用可能性も高く、今後のさらなる展開が期待される。

謝辞

本研究では, 産業技術総合研究所の AI 橋渡しクラウド (ABCI 3.0) の計算資源を利用した. また, コンペティションを主催された国立情報学研究所 (NII) に感謝する.

参考文献

- [1] J. Wei ほか, 「Chain-of-Thought Prompting Elicits Reasoning in Large Language Models」, Advances in Neural Information Processing Systems, 2022, pp. 24824–24837.
- [2] S. Toshniwal と others, 「OpenMathInstruct-1: A 1.8 Million Math Instruction Tuning Dataset」, arXiv preprint arXiv:2402.10176, 2024.
- [3] Z. Shao ほか, 「DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models」, arXiv preprint arXiv:2402.03300, 2024.
- [4] D. Guo ほか, 「DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning」, arXiv preprint arXiv:2501.12948, 2025.
- [5] X. Wang ほか, 「Self-Consistency Improves Chain of Thought Reasoning in Language Models」, The Eleventh International Conference on Learning Representations, 2023.
- [6] N. Muennighoff と others, 「s1: Simple Test-Time Scaling」, arXiv preprint arXiv:2501.19393, 2025.